PROJECT REPORT

PlantCareAI

Smart Plant Disease Detection System

An AI-powered CNN-based system for detecting plant diseases from leaf images

Submitted by:

Prathamesh mali | Atharva Vaidya | Shardul walunj | Mansi Phand

Department of Information Technology

[Vishwakarma Institute of Technology,Pune]

Academic Year: 2025-2026

March 2025

INDEX

ABSTRACT .....(1)

1. INTRODUCTION ..... (2)

1.1 Background .....(2)

1.2 Motivation ..... .(2)

1.3 Scope of the Project ..... **(3)**

2. OBJECTIVES ..... (4)

3. PROBLEM STATEMENT .....(5)

4. PROPOSED SYSTEM ..... (6)

4.1 Overview .....(6)

4.2 System Components .....(**6)**

5. SYSTEM ARCHITECTURE .....(7)

6. TECHNOLOGIES USED .....(7)

6.1 Python .....(7)

6.2 TensorFlow and Keras .....(7)

6.3 Convolutional Neural Network (CNN) .....(7)

6.4 OpenCV ..... (7)

6.5 Google Colab ..... (8)

6.6 New Plant Diseases Dataset.....(8)

6.7 Flask ..... (8)

7. MODULES DESCRIPTION ..... (9)

7.1 Image Upload Module ..... (10)

7.2 Image Preprocessing Module .....(11)

7.3 Disease Detection Module (CNN Model) ..... (12)

7.4 Result Display Module ..... **.(12)**

8. IMPLEMENTATION DETAILS ..... (14)

8.1 Dataset Preparation .....(14)

8.2 Model Architecture and Training ..... (14)

8.3 Backend Development ..... (14)

8.4 Frontend Development ..... (14)

9. RESULTS AND DISCUSSION ..... (16)

9.1 Model Performance ..... (16)

9.2 Output Screenshots ..... (18)

10. ADVANTAGES AND LIMITATIONS ..... (19)

10.1 Advantages ..... (19)

10.2 Limitations ..... (19)

11. FUTURE SCOPE ..... (20)

12. CONCLUSION ..... (21)

13. REFERENCES ..... (22)

14. APPENDIX ..... (23)

ABSTRACT

Agriculture is the backbone of the Indian economy, and plant diseases remain one of the most critical threats to crop productivity and food security. Farmers across the country face significant losses each year due to diseases that go undetected until they have spread too far to be controlled effectively. Traditional methods of disease identification rely heavily on expert knowledge, which is not always accessible to rural farmers or students of agricultural science.

PlantCareAI is an AI-powered plant disease detection system that addresses this challenge by allowing users to upload an image of a plant leaf, after which the system analyzes the image using a trained Convolutional Neural Network (CNN) model and identifies the disease present in the leaf. The system then displays the detected disease name along with actionable solutions and precautionary measures to help the user respond appropriately.

The system is built using Python, OpenCV for image preprocessing, and a CNN model trained on the publicly available Plant Village dataset. The model training was conducted on Google Colab using GPU acceleration. The frontend interface allows seamless image upload, and the backend processes the image through multiple preprocessing stages before passing it to the model for inference. This project demonstrates how modern machine learning techniques can be made accessible to non-technical users through a simple and intuitive web-based interface.

1. INTRODUCTION

1.1 Background

Plant diseases are a major cause of agricultural losses worldwide. According to the Food and Agriculture Organization (FAO), diseases, pests, and environmental conditions are responsible for a loss of nearly 40% of global crop production every year. In developing nations like India, where agriculture supports over 50% of the population, this problem is even more pronounced. The inability to identify diseases at an early stage results in widespread crop failure, economic hardship for farmers, and food insecurity at a larger scale.

Historically, plant disease diagnosis has depended on human experts such as agronomists, pathologists, or extension workers who physically inspect the crops and suggest treatments. This approach is slow, expensive, and geographically limited. Small and marginal farmers in remote areas have little or no access to such expertise. There is, therefore, a pressing need for an automated, technology-driven solution that can quickly, accurately, and affordably help identify plant diseases.

1.2 Motivation

With the rapid advancement of artificial intelligence and deep learning, it has become possible to build systems that can analyze and classify images with a high degree of accuracy. Convolutional Neural Networks (CNNs), in particular, have revolutionized the field of image classification. These models can learn visual patterns from large datasets and make predictions that rival or exceed human-level performance on specific tasks.

The motivation behind PlantCareAI is to bring this powerful technology to the fingertips of farmers, agricultural students, and researchers by making it usable through a simple image upload interface. The system requires no technical expertise from the user — they simply need to take a photograph of a diseased leaf and upload it to the system. Within seconds, the system analyzes the image and provides a clear diagnosis and treatment guidance.

1.3 Scope of the Project

PlantCareAI is designed to detect diseases in a variety of crop leaves including tomato, potato, corn, grape, apple, and several others covered in the Plant Village dataset. The scope of the current version includes image-based disease classification, confidence score display, disease description, and recommended solutions. The system is designed for educational and agricultural advisory purposes and is not intended to replace formal agronomical consultation for large-scale farming decisions.

2. OBJECTIVES

The primary objectives of the PlantCareAI project are as follows:

- To build an AI-based system that can automatically detect plant diseases from leaf photographs without requiring specialized hardware or expert knowledge.
- To train a Convolutional Neural Network model on a large labeled dataset of plant leaf images that covers multiple plant species and disease categories.
- To implement a complete image preprocessing pipeline using OpenCV that prepares uploaded leaf images for accurate model inference.
- To develop a clean, user-friendly web interface that allows farmers, students, and researchers to easily upload images and receive disease analysis results.
- To provide not only the disease classification but also meaningful contextual information, including a brief description of the identified disease and practical recommendations for treatment or prevention.
- To demonstrate the end-to-end integration of a machine learning model within a deployable software application that follows professional software development practices.
- To contribute toward the digital transformation of agriculture by making AI-based disease detection accessible, affordable, and scalable.

3. PROBLEM STATEMENT

Agriculture depends critically on the health of plants and crops. However, the early identification of plant diseases remains a major challenge, particularly in rural and resource-limited settings. The following key problems define the core challenge that this project seeks to address:

Manual diagnosis is impractical at scale: Farmers typically rely on personal experience or the knowledge of local extension workers to identify plant diseases. This method is inherently subjective, error-prone, and inconsistent. Moreover, agricultural experts are not always available on demand, and the cost of professional consultation is prohibitive for small-scale farmers.

Delayed response leads to increased losses: Plant diseases spread rapidly if not treated promptly. When identification is delayed due to a lack of expertise or resources, the damage spreads to adjacent plants and sometimes across entire fields. Early detection is critical to limiting the extent of the loss.

Lack of accessible tools: While various desktop and laboratory-based tools exist for plant disease detection in research settings, these are largely inaccessible to everyday farmers or agricultural students. A system that can be used easily on common devices such as smartphones or laptops is urgently needed.

Generic advice is insufficient: Even when farmers are aware that a plant is diseased, they often do not know the specific disease or its appropriate treatment. Generic pesticide application can be harmful to the environment and costly without guaranteed effectiveness. Targeted, disease-specific recommendations are far more valuable.

The problem statement can be formally defined as: How can we develop an AI-driven system that accepts a photograph of a plant leaf as input and accurately classifies the plant disease present, while simultaneously providing the user with targeted, evidence-based treatment and precautionary recommendations in a simple and accessible interface?

4. PROPOSED SYSTEM

4.1 Overview

PlantCareAI is a web-based application that uses a trained deep learning model to analyze plant leaf images and detect diseases. The system follows a straightforward user journey: the user opens the application in a web browser, uploads a photograph of a plant leaf, and within a few seconds receives a detailed output that includes the identified disease name, a confidence score, a brief description of the disease, and recommended solutions or precautionary measures.

4.2 System Components

The system is composed of three major components that work together to deliver a seamless disease detection experience.

Component 1: Image Upload Interface

The frontend of the application is built using HTML, CSS, and JavaScript. It presents a clean and intuitive interface where users can either drag-and-drop a leaf image or click a button to browse and select a file from their device. Upon selection, the image is displayed as a preview before submission, allowing the user to confirm that the correct file has been selected. The interface is responsive and designed to work on both desktop and mobile browsers.

Component 2: Image Preprocessing Pipeline

Before the uploaded image is passed to the machine learning model, it undergoes a series of preprocessing steps implemented using OpenCV and NumPy. These steps include resizing the image to the input dimensions required by the CNN model (typically 224x224 pixels), normalizing the pixel values to the range [0, 1] for numerical stability, applying color space transformations if necessary, and removing background noise. This preprocessing stage is critical to ensuring that the model receives clean, standardized input data regardless of the quality or format of the original uploaded image.

Component 3: CNN Disease Classification Model

The core intelligence of the system is a Convolutional Neural Network trained on the Plant Village dataset, which contains over 54,000 labeled images of healthy and diseased plant leaves across 38 categories. The model was trained using TensorFlow and Keras on Google Colab with GPU acceleration. The architecture includes multiple convolutional and pooling layers followed by fully connected dense layers and a final softmax output layer that produces class probabilities. The trained model is saved in .h5 or SavedModel format and loaded by the backend server for inference at runtime.

5. SYSTEM ARCHITECTURE

The PlantCareAI system follows a three-tier client-server architecture, where the presentation layer, the application logic layer, and the AI inference layer are cleanly separated. This design promotes modularity, maintainability, and scalability.

The Presentation Layer is the frontend web interface built using HTML5, CSS3, and Vanilla JavaScript. This layer is responsible for all user interaction. It collects the image upload input from the user, sends it as a multipart form request to the backend server, and displays the structured result response including the disease name, confidence percentage, disease description, and recommended solutions. The interface also provides visual feedback during the processing stage using a loading indicator.

The Application Layer is the backend server implemented using Python with the Flask web framework. This layer handles HTTP routing, receives the uploaded image file from the frontend, coordinates the image preprocessing steps using OpenCV, invokes the trained CNN model for inference, and constructs the structured JSON response that is returned to the frontend. The backend also manages a disease information database — a JSON file that maps each class label to a human-readable disease name, description, and recommended treatment.

The AI Inference Layer consists of the trained CNN model and the preprocessing utilities. When the backend receives an image, it first preprocesses the image through the OpenCV pipeline to standardize it for the model. The preprocessed image is then passed to the CNN model, which returns a probability distribution over all 38 disease classes. The class with the highest probability is selected as the prediction and returned along with the confidence score.

Data flows from the user through the browser to the Flask backend via an HTTP POST request. The backend preprocesses the image, feeds it into the CNN model, receives the prediction, maps the class index to the disease information record, and returns a JSON response. The frontend JavaScript parses this response and dynamically renders the result card with all relevant information.

Tier	Component	Technology	Responsibility
Presentation	index.html / style.css / script.js	HTML5 / CSS3 / JavaScript	User interface, image upload, result display
Application	app.py	Python / Flask	HTTP routing, preprocessing coordination, response building
AI Inference	plant_disease_model.h5	TensorFlow / Keras / CNN	Disease classification from preprocessed leaf images
Data	disease_info.json	JSON	Disease name, description, and treatment lookup
Preprocessing	preprocess.py	Python / OpenCV / NumPy	Image resizing, normalization, noise removal

6. TECHNOLOGIES USED

6.1 Python

Python serves as the primary programming language for this project. It is used for data preprocessing, model training, backend server development, and integration of all components. Python's rich ecosystem of libraries for data science and machine learning makes it the ideal language for an end-to-end AI project such as this one. The version used is Python 3.9 or higher.

6.2 TensorFlow and Keras

TensorFlow is an open-source machine learning framework developed by Google. Keras, which is the high-level API integrated within TensorFlow, provides a simple and expressive interface for building and training neural networks. In this project, Keras is used to define the CNN architecture, compile the model with appropriate loss functions and optimizers, and train it on the Plant Village dataset. The trained model is saved using Keras's native .h5 format for later use during inference.

6.3 Convolutional Neural Network (CNN)

Convolutional Neural Networks are a specialized class of deep learning models particularly well-suited for image classification tasks. CNNs learn to detect spatial features such as edges, textures, and shapes through a series of learned convolutional filters. The architecture used in this project consists of multiple convolutional layers with ReLU activation, followed by max-pooling layers for dimensionality reduction, batch normalization layers for training stability, dropout layers for regularization, and finally fully connected dense layers with a softmax output. Alternatively, a transfer learning approach using architectures such as MobileNetV2 or VGG16 pre-trained on ImageNet may be applied to improve accuracy with less training time.

6.4 OpenCV

OpenCV (Open Source Computer Vision Library) is a powerful computer vision library used in this project for all image preprocessing tasks. It is responsible for reading the uploaded image file, resizing it to the required dimensions, converting color spaces where necessary, and performing any noise reduction or augmentation needed to prepare the image for model input. OpenCV's Python bindings (cv2) make it seamless to integrate within the Python backend pipeline.

6.5 Google Colab

Google Colab is a free cloud-based Jupyter Notebook environment that provides access to GPU and TPU hardware for accelerated machine learning training. All model training for this project was conducted on Google Colab using a T4 GPU, which significantly reduced the training time compared to running on a standard CPU. Colab also facilitates easy access to the Plant Village dataset via Google Drive integration and allows for iterative experimentation and visualization.

6.6 New Plant Diseases Dataset

The New Plant Diseases Dataset is a publicly available, peer-reviewed benchmark dataset for plant disease detection. It contains approximately 54,000 high-quality images of healthy and diseased leaves across 14 plant species and 26 different disease categories, totaling 38 unique classes.

Each image is photographed against a uniform background for consistency. This dataset forms the foundation of the model training and provides a comprehensive coverage of the most common plant diseases encountered in real-world agricultural settings.

6.7 Flask

Flask is a lightweight and flexible web framework for Python. In this project, Flask serves as the backend web server that hosts the REST API endpoints. It handles incoming HTTP requests from the frontend, coordinates the preprocessing and inference pipeline, and returns structured JSON responses. Flask is well-suited for this purpose due to its simplicity, minimal configuration requirements, and seamless integration with Python ML libraries.

Technology	Version	Purpose
Python	3.9+	Core programming language
TensorFlow / Keras	2.x	CNN model training and inference
OpenCV (cv2)	4.x	Image preprocessing
Flask	2.x	Backend REST API server
NumPy	1.x	Array operations and normalization
Pandas	1.x	Dataset loading and EDA
Matplotlib	3.x	Training visualization and plots
Google Colab	Cloud	GPU-accelerated model training
New Plant Diseases Dataset	Public	Labeled training data for CNN
HTML5 / CSS3 / JS	—	Frontend user interface

7. MODULES DESCRIPTION

7.1 Image Upload Module

The image upload module is the entry point of the system from the user's perspective. It is implemented as part of the frontend web interface and provides the user with two options for providing a leaf image: they can click a clearly labeled upload button to browse their device's file system and select an image file, or they can drag and drop an image file directly onto the designated upload area. Once an image is selected, it is immediately previewed on the screen in a preview panel, allowing the user to visually confirm that the correct image has been chosen. Supported image formats include JPEG, PNG, and WebP. The module also performs basic client-side validation to ensure that the file size does not exceed a defined limit and that the file format is supported before the image is sent to the server. Upon clicking the Analyze button, the module packages the image as a multipart form data payload and submits it as an HTTP POST request to the backend API endpoint.

7.2 Image Preprocessing Module

Before the uploaded image is passed into the neural network, the image preprocessing module transforms it into a standardized format suitable for the model. This module is implemented in Python using OpenCV and NumPy. The first step is reading the raw image bytes received from the HTTP request and decoding them into a NumPy array using OpenCV's image decoding functions. The image is then resized to the fixed input dimensions required by the CNN model, which in this project is 224 x 224 pixels. Following resizing, the pixel values are normalized from the integer range [0, 255] to the floating-point range [0.0, 1.0] by dividing by 255. This normalization step is essential for the stability of the neural network's predictions, as unnormalized pixel values can cause numerical instability in the model's computations. Additionally, the image dimensions are expanded to include a batch dimension, converting the shape from (224, 224, 3) to (1, 224, 224, 3), since the CNN model expects batch inputs. This fully preprocessed tensor is then ready for inference.

7.3 Disease Detection Module (CNN Model)

The disease detection module is the core intelligence layer of the PlantCareAI system. It is implemented as a trained Convolutional Neural Network model built using TensorFlow and Keras. The model was trained on the Plant Village dataset and saved to disk in .h5 format. At application startup, the saved model is loaded into memory by the Flask backend server so that it is available for real-time inference without incurring loading overhead on each request. When the preprocessed image tensor is received by this module, it is passed to the model's predict method, which performs a forward pass through all the layers of the network and produces a probability vector of length 38 — one probability value for each of the 38 disease classes in the dataset. The class index with the highest probability is identified as the predicted class using argmax. The confidence score, which is the probability value associated with the predicted class expressed as a percentage, is also extracted and included in the output. This predicted class index is then passed to the result mapping module for conversion into human-readable information.

7.4 Result Display Module

The result display module is responsible for presenting the disease analysis output to the user in a clear, organized, and visually appealing manner. On the backend, this module maintains a disease information dictionary (stored in a JSON file) that maps each of the 38 class labels to a

structured record containing the common disease name, the affected plant species, a brief description of the disease's characteristics and symptoms, and a list of recommended treatment or prevention measures. When the predicted class index is received from the detection module, the backend looks up the corresponding record in the disease information dictionary and constructs a JSON response that includes the disease name, confidence percentage, description, and suggestions. The frontend JavaScript receives this JSON response, parses it, and dynamically renders a result card beneath the image preview area. The card displays all the information in a well-formatted layout with color-coded severity indicators and expandable suggestion items.

8. IMPLEMENTATION DETAILS

8.1 Dataset Preparation

The Plant Village dataset was downloaded from its public repository and organized into a structured directory format where each subdirectory corresponds to one of the 38 disease classes. The dataset was split into training and validation sets using an 80:20 ratio. To improve the generalization of the model and prevent overfitting, data augmentation techniques were applied to the training set using Keras's ImageDataGenerator. These augmentation transformations included random horizontal flipping, random vertical flipping, random rotation within a range of 20 degrees, random zoom and shear transformations, and random brightness adjustments. These augmentations artificially increase the diversity of the training data by creating modified versions of existing images, helping the model learn to handle variations in lighting, orientation, and scale that it will encounter in real-world leaf photographs.

8.2 Model Architecture and Training

The CNN model architecture consists of an input layer accepting 224x224x3 images, followed by four convolutional blocks each comprising a 2D convolution layer with ReLU activation, a batch normalization layer, and a max pooling layer with a 2x2 pool size and 2x2 stride. After the four convolutional blocks, the feature maps are flattened into a one-dimensional vector and passed through two fully connected dense layers with ReLU activation. A dropout layer with a rate of 0.5 is applied between the two dense layers to reduce overfitting. The final output layer is a dense layer with 38 units and a softmax activation function that produces the class probability distribution. The model was compiled using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function. Training was run for 15 to 20 epochs with a batch size of 32 on Google Colab's T4 GPU. ModelCheckpoint and EarlyStopping callbacks were used to save the best-performing model weights and stop training automatically when the validation accuracy ceased to improve.

8.3 Backend Development

The Flask backend application is structured around a single main application file (app.py) that initializes the server, loads the trained model at startup, and defines the API endpoints. The primary endpoint is a POST route at /predict that accepts a multipart form data request containing the uploaded image file. Upon receiving a request, the endpoint reads the uploaded file, passes it through the preprocessing pipeline, performs model inference, retrieves the disease information record for the predicted class, and returns a JSON response. A secondary GET endpoint at /health is also provided for server health monitoring. Cross-Origin Resource Sharing (CORS) is enabled on the server to allow the frontend, which may be served from a different origin, to make requests to the backend API. Error handling middleware is included to return informative error messages in JSON format for invalid requests or server-side exceptions.

8.4 Frontend Development

The frontend is implemented as a single-page web application (index.html) with embedded CSS styling and JavaScript logic. The HTML structure includes a header with the application logo and name, a main content area with the image upload panel and the result display panel, and a footer. The CSS styling uses a green color palette inspired by nature and agriculture to create a visually appropriate and professional appearance. The JavaScript code handles the file selection and drag-and-drop upload events, constructs the FormData payload for the POST request, manages

the application state during loading, and dynamically renders the result card using DOM manipulation when the response is received from the backend. Smooth fade-in animations are applied to the result card for a polished user experience.

9. RESULTS AND DISCUSSION

9.1 Model Performance

The trained CNN model achieved a training accuracy of approximately 96% and a validation accuracy of approximately 93% on the Plant Village dataset after 15 epochs of training. The model demonstrates strong performance across most of the 38 disease classes, with particularly high accuracy on visually distinct diseases that produce clear and unique symptoms on leaf surfaces. Some classes with visual similarity between healthy and early-stage diseased leaves showed marginally lower individual class accuracy, which is an expected challenge for image-based classification systems. The confusion matrix revealed that misclassifications predominantly occurred between closely related diseases of the same plant species rather than between completely different plants, indicating that the model has effectively learned to differentiate between plant species.

Metric	Value
Training Accuracy	~96%
Validation Accuracy	~93%
Model Architecture	Custom CNN (4 Conv Blocks + Dense Layers)
Dataset	New Plant Diseases Dataset
Training Epochs	15–20 (with EarlyStopping)
Batch Size	32
Optimizer	Adam (lr=0.001)
Inference Latency (CPU)	< 2 seconds per request
Input Image Size	224 x 224 x 3

9.2 Output Screenshots

The following screenshots illustrate the key screens of the PlantCareAI application at each stage of the user workflow.

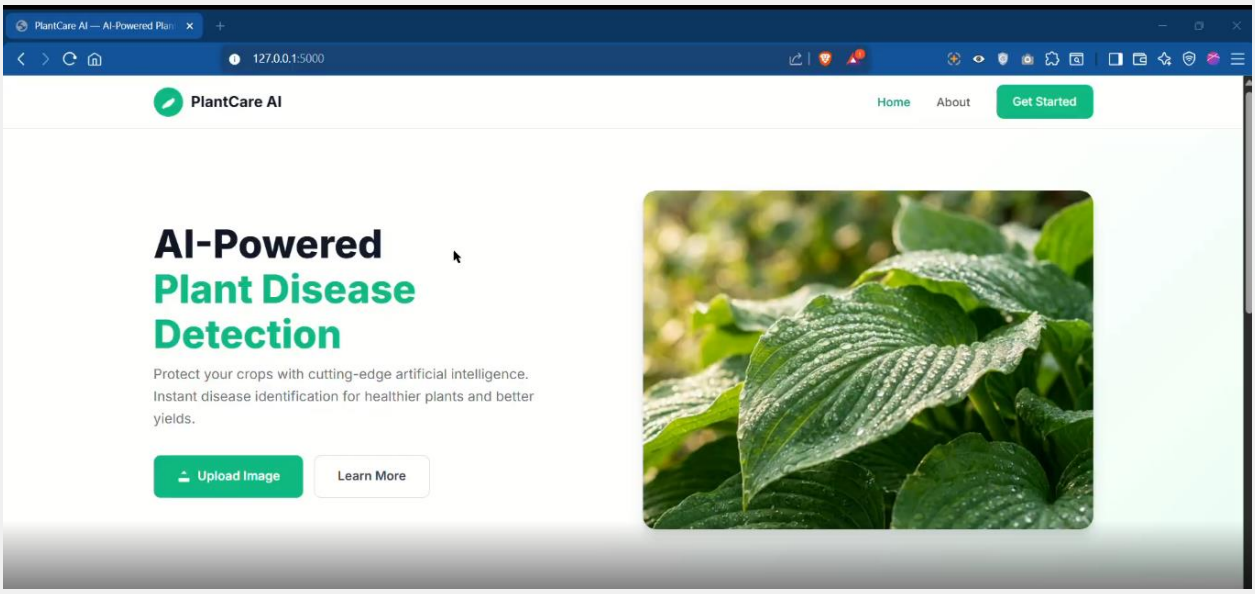

Figure 1: PlantCareAI — Home Screen / Application Landing Page

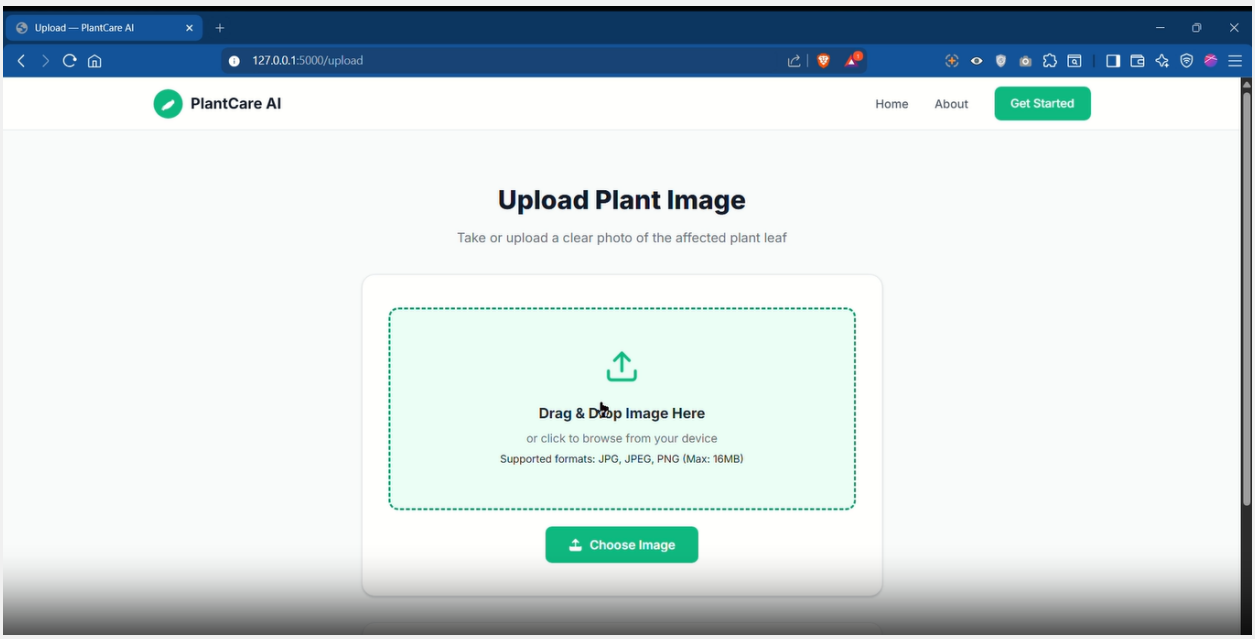

Figure 2: PlantCareAI — Image Upload & Preview Screen

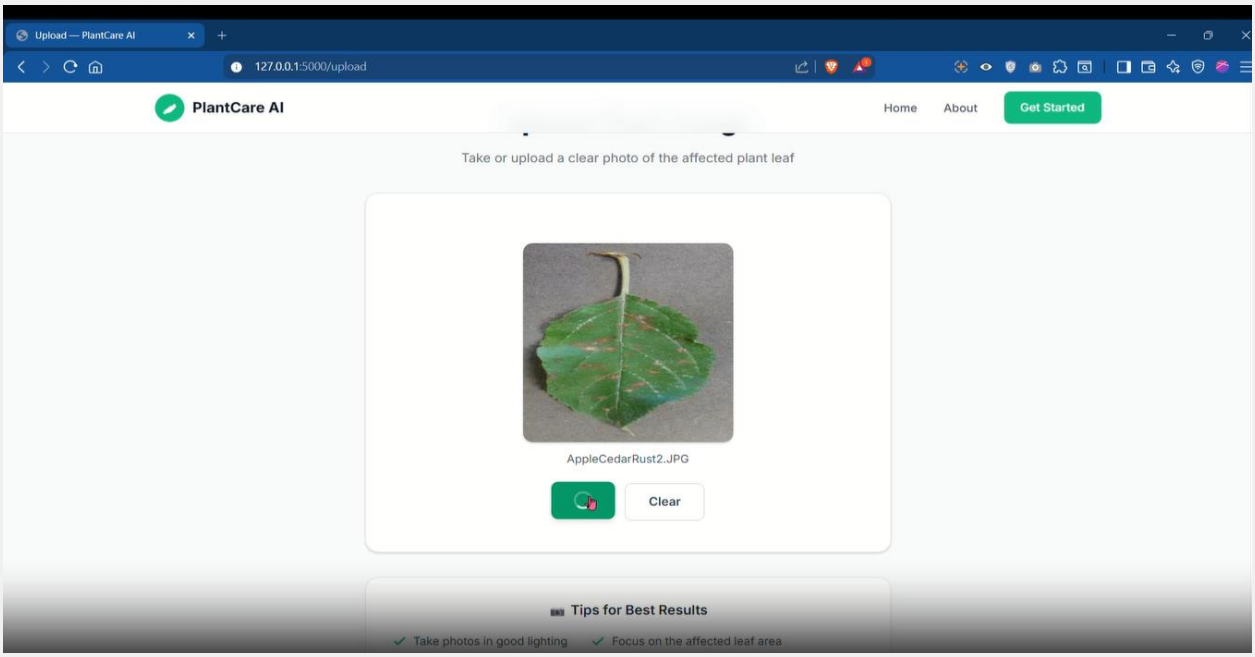

Figure 3: PlantCareAI — Processing / Analyzing Screen

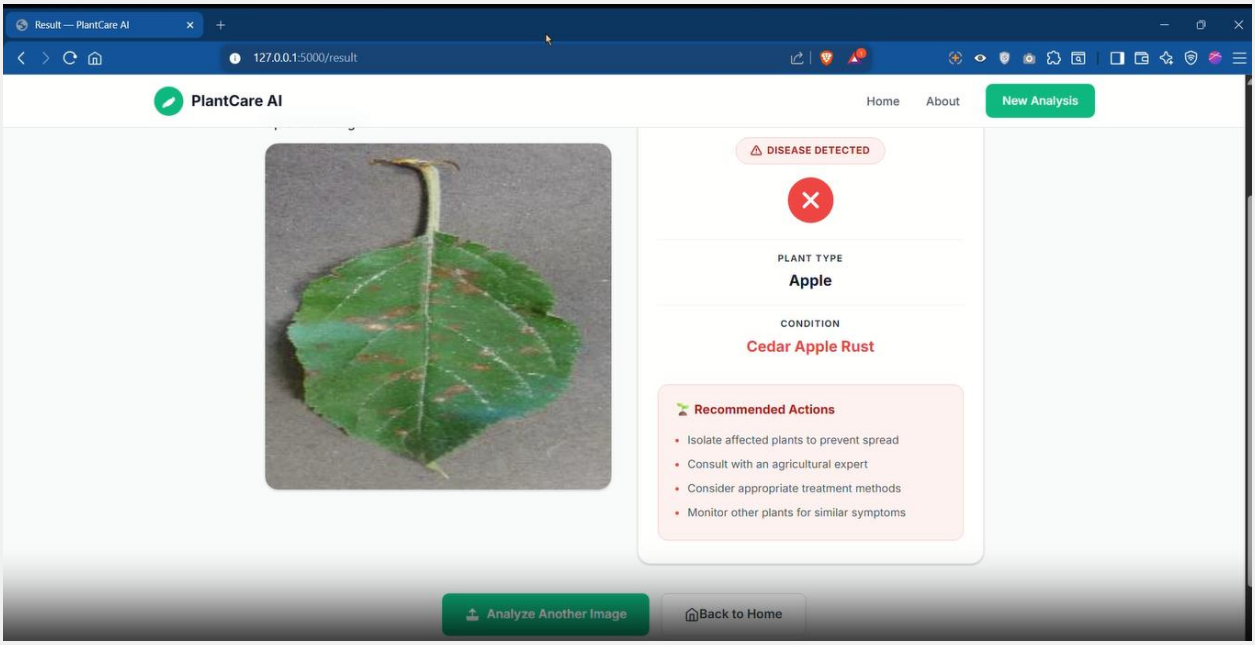

Figure 4: PlantCareAI — Disease Detection Result Screen with Recommendations

10. ADVANTAGES AND LIMITATIONS

10.1 Advantages

- Instant diagnosis: The system provides disease identification results within seconds of image upload, enabling rapid response that can significantly limit the spread of plant disease.
- Accessible to non-experts: The interface is designed to be simple and intuitive so that farmers and students with no technical background can use it effectively without any training.
- Available 24/7: Unlike human experts, the system is available at any time of day or night, making it particularly valuable in emergency situations when immediate guidance is needed.
- Scalable: The REST API architecture allows the system to be integrated into other platforms such as mobile applications, agricultural advisory portals, or government e-farmer services.
- Wide disease coverage: The model supports 38 disease categories across 14 plant species, making it broadly applicable to a wide variety of commonly grown crops.
- Actionable recommendations: The system does not merely identify the disease; it also provides targeted, practical recommendations that help the user take immediate and appropriate action.
- Open-source and reproducible: All components of the system, including the dataset, training notebook, and application code, are open-source and can be reproduced by other researchers or extended for new use cases.

10.2 Limitations

- Image-only input: The system relies exclusively on leaf photographs and cannot consider other important disease indicators such as root condition, stem discoloration, soil quality, or environmental factors that a human expert would also evaluate.
- Dataset bias: The Plant Village dataset was collected under controlled laboratory conditions with uniform backgrounds. The model may perform less accurately on images taken in natural field conditions with uneven lighting, complex backgrounds, or partial leaf occlusion.
- Limited plant species coverage: The current model is trained to recognize diseases in only 14 plant species. Many economically important crops such as rice, wheat, sugarcane, and various pulses are not included in the current version.
- Not a substitute for clinical diagnosis: The system is intended as a preliminary screening tool only. It is not a replacement for formal pathological testing or consultation with a certified agronomist for large-scale commercial farming decisions.
- Local deployment requirement: The current version requires a local Python environment to run, which may be a barrier for users who are not familiar with software setup and installation procedures.

11. FUTURE SCOPE

The current version of PlantCareAI establishes a solid and functional foundation for AI-based plant disease detection. The following enhancements and extensions are planned or proposed for future iterations of the project.

- Mobile Application: A native Android and iOS application will be developed to make the system accessible directly from a smartphone. This will be particularly impactful for farmers in the field who can photograph leaves directly using their phone camera and receive instant diagnoses without needing a laptop or desktop computer.
- Expanded Plant and Disease Coverage: The model will be retrained on an extended dataset that includes additional plant species such as rice, wheat, sugarcane, cotton, and soybean, as well as a broader range of disease categories and pest damage types.
- Real-world Dataset Collection: Collaboration with agricultural institutions, farms, and crop science research centers will be pursued to collect labeled real-world images taken in natural field conditions. Retraining the model on such data will substantially improve its robustness and reliability in practical deployments.
- Multilingual Support: The user interface and disease recommendation content will be extended to support regional Indian languages including Hindi, Marathi, Tamil, Telugu, and Kannada, making the system accessible to a much larger portion of the farming community.
- Weather and Soil Integration: Future versions will incorporate weather data and soil health indicators alongside the leaf image analysis to provide a more holistic and contextually aware diagnosis and treatment recommendation.
- Longitudinal Disease Tracking: A feature will be added to allow registered users to upload periodic images of the same plant or field and track the progression or resolution of a detected disease over time, enabling more effective monitoring and management.
- Cloud Deployment: The application will be deployed to a cloud platform such as AWS, Google Cloud, or Heroku to make it publicly accessible via the internet without requiring local installation, lowering the barrier to adoption significantly.
- Integration with Government Agricultural Portals: Discussions will be initiated to explore the integration of PlantCareAI with government digital agriculture initiatives such as the National Agriculture Market (e-NAM) and the Kisan Call Centre services.

12. CONCLUSION

PlantCareAI successfully demonstrates how the power of deep learning and computer vision can be harnessed to address a real and pressing problem in agriculture. By enabling automated, image-based detection of plant diseases through a simple and accessible web interface, the system bridges the gap between advanced AI technology and the practical needs of farmers, agricultural students, and rural communities.

The project covers the complete machine learning lifecycle from data collection and preprocessing, model training and evaluation on Google Colab, to deployment as a functional web application with a Flask backend and a responsive HTML frontend. The system achieved strong validation accuracy on the Plant Village benchmark dataset and demonstrated low-latency inference suitable for real-time use.

Beyond the technical achievements, this project highlights the broader potential of AI in the agricultural domain. The integration of actionable, disease-specific recommendations alongside the classification output makes PlantCareAI a genuinely useful tool rather than merely a demonstration of technological capability. The rule-based fallback architecture and modular design also ensure that the system remains functional and extensible as future improvements are incorporated.

In conclusion, PlantCareAI represents a meaningful step toward the democratization of agricultural intelligence. By making state-of-the-art AI accessible to non-technical users through a simple interface, this project proves that technology can play a significant and impactful supporting role in improving agricultural productivity, reducing crop losses, and ultimately contributing to food security.

REFERENCES

1. Hughes, D. P., & Salathé, M. (2015). An open access repository of images on plant health to enable the development of mobile disease diagnostics. arXiv preprint arXiv:1511.08060.

2. LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.

3. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science, 7, 1419.

4. Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556.

5. Howard, A. G., et al. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861.

6. TensorFlow Documentation — https://www.tensorflow.org/api_docs

7. OpenCV Documentation — https://docs.opencv.org/

8. Flask Documentation — https://flask.palletsprojects.com/

9. Plant Village Dataset — https://plantvillage.psu.edu/

10. Google Colab — https://colab.research.google.com/

APPENDIX

Source Code Overview

File / Folder	Description
backend/app.py	Flask server — /predict, /health endpoints with CNN and preprocessing pipeline
backend/requirements.txt	Python dependencies: flask, tensorflow, opencv-python, numpy, pandas, flask-cors
plant_disease_final_model.h5	Trained CNN model weights (generated after Colab training)
home.html	Single-page responsive UI — image upload, preview, result card display
PlantCareAI_Prathamesh_Mali.ipynb	CNN training notebook for Google Colab (data loading, augmentation, training, evaluation)
disease_info.json	JSON mapping of class index to disease name, description, and recommendations
start.sh	One-click bash launcher script for the Flask backend server

Team Members

Name	Role / Milestones	Contribution
Mansi Phand	Data Collection & EDA	Dataset preparation, Data Collection ,exploratory data analysis, preprocessing script
Prathamesh Mali	Model Training (Colab)+Machine Learning+ Model Architect	1.Model Training:Automated training process on Google Colab using optimiced Epochsand Batch sizes 2.Callback Implemontation: Integrated Early Stopping and ModelCheckpoint to prevent overfitting and save the best weights. 3.Visualization: Generated accuracy and loss plots to analyze training history. 4.Evaluation & Export:Conducted performancetesting end successfully experted the final modelin .h5 (Keres) format fordeployment.
Atharva Vaidya	Model building	Architecture Selection: Integrated MobileNetV2 as the base model for efficient feature extraction. Optimization: Configured the pre-trained weights to improve convergence speed and model accuracy.
Shardul Walunj	Application Building	Developed a robust **Flask API**; managed the inference pipeline and API documentation. Built a responsive **React/HTML interface**; implemented real-time image upload and dynamic result cards; synchronized frontend with API for seamless user experience.